

Paradigmes de Programmation

M1/GL/S1/2025/2026

Série N° 03 (Paradigme fonctionnel)

(Durée: 2H30.)

Exercice 1.

1. Définir une fonction *abss* qui calcule la valeur absolue d'un nombre entier, par exemple *abss*(-2) évaluée à 2.
2. Définir une fonction *pgcd* qui calcule le plus grand commun diviseur de deux nombres entiers. (Utiliser la fonction *abss* définie précédemment pour gérer les nombres négatifs,),
3. On considère un nombre rationnel comme une paire (*num*, *den*) de deux nombres entiers, représentant une fraction. Écrire une fonction *norm* qui normalise une fraction, c'est-à-dire *norm* réduit une fraction telle que numérateur et dénominateur n'ont pas de facteurs en commun. Par exemple *norm* (4, 6) évalue à (2,3).
4. Définir deux fonctions *somme* et *produit* qui calculent, respectivement, la somme et le produit de deux fractions. Le résultat doit être une fraction normalisée.

Exercice 2

Écrire les fonctions Ocaml suivantes et donner leurs types.

1. ***nieme*** : Renvoie le n-ième élément d'une liste ;
2. ***lg*** : Renvoie la longueur de la liste ;
3. ***concat*** : Renvoie la concaténation de deux listes ;
4. ***inverse*** : Renvoie la liste en ordre inversé ;
5. ***applatit*** : Concatène les éléments d'une liste de listes ;
6. ***dernier*** : Renvoie le dernier élément d'une liste non vide ;
7. ***appartient*** : Renvoie true si l'élément appartient à la liste, false sinon ;
8. ***zip*** : prend deux listes et renvoie une liste de paires associant les éléments correspondants. Si les deux listes n'ont pas la même longueur, on s'arrête dès que l'une des deux est vide.

Exemple : *zip* [1; 2; 3] ['a'; 'b'; 'c'] ;; évaluée à [(1, 'a'); (2, 'b'); (3, 'c')]